

copperbench: Benchmarking Language Models on Verified Hardware Design Edits

Animesh Chouhan
Chouhan Industries
animeshchouhan@outlook.com

September 7, 2026

Suite version 0.1.0-draft Snapshot none 0 runs

Abstract

Language-model agents are now evaluated extensively on software engineering, where a test suite supplies the oracle. Hardware design presents a different structure: strong deterministic checkers exist (electrical rule checks, design rule checks, netlist-to-document consistency), but there is no equivalent of a unit test that states what the change was supposed to accomplish, and an error is discovered at a fabrication run rather than at a rerun. We present *copperbench*, a benchmark that evaluates agents on real KiCad projects through the copperhead agent harness, using electrical verification as a partial oracle and pairing it with end-state assertions that state what the requested change was.

The benchmark makes three commitments that we argue any agentic benchmark in this setting needs. First, grading is deterministic, offline, and model-free: reproducing a published score requires the repository but no provider credential. Second, tasks are declarative data graded by a closed assertion vocabulary, so two tasks are comparable by construction rather than by the good intentions of whoever wrote their checkers. Third, the suite contains tasks whose correct outcome is a refusal, drawn from budget arithmetic that the requested change would violate, alongside tasks with a correct edit, so that neither compliance nor caution is a winning strategy on its own.

We report strict pass rate and cost per passing task, separately per difficulty tier, along with consistency across repeats, refusal correctness, edit surgicality, and a deterministic failure taxonomy that ranks what to fix next. This revision covers none across 2 tasks and 0 runs at 3 repeats; api-frontier, api-cheap, open-weight, saved-login remain uncovered and are labeled as such throughout. Every figure in this paper is generated from the released result records, and a claim not backed by a generated value fails the build.

1 Introduction

Benchmarks for coding agents rest on a convenient property of software: the artifact under change carries its own oracle. SWE-bench and its descendants can ask whether a patch makes a failing test pass, because the test was written by a human who knew what correct meant [Jimenez et al., 2024]. The agent’s output is executable, the oracle is executable, and the loop closes in seconds.

Hardware design does not have that property, and the ways it lacks it are more interesting than the fact that it does.

The oracle is strong but silent about intent. A KiCad project ships with real checkers. Electrical rule checking finds unconnected pins, conflicting drivers, and unpowered nets. Design rule checking finds clearance and width violations. These are fast, deterministic, and free of the model. What none of them will tell you is whether the agent did the thing that was asked. A schematic in which nothing changed passes every check. So does one in which the agent renamed the wrong net. The verification layer establishes that a design is *legal*, and legality is orthogonal to intent.

The failure mode is silent compliance, not a crash. In software, an agent that misunderstands a task usually produces something that fails loudly. In hardware, the characteristic failure is an edit that is electrically valid, internally consistent, documented, committed, and wrong. A pullup resistor that violates a sleep-current budget passes every check the project can run. It is caught by arithmetic against a recorded constraint or it is not caught at all, and it is discovered on a board that has already been fabricated.

The correct answer is sometimes refusal. Because designs carry budgets that cannot all be satisfied at once, some well-formed requests have no correct implementation. An agent that complies with all of them is not more capable than one that refuses some; it is less useful and more dangerous. This makes refusal a first-class behavior to measure, which in turn makes it a behavior that can be gamed, which in turn constrains how a suite must be built.

Runs are expensive and non-deterministic. An agent run on a real project takes minutes and costs tokens. A design that needs hundreds of runs before it says anything is a design that will be executed once and then quoted forever.

1.1 Contributions

- **A benchmark over real KiCad projects** with electrical verification as a partial oracle, paired with end-state assertions that encode what the requested change was, so that legality and intent are graded separately.
- **A deterministic, offline grading contract.** Every score is recomputable from three artifacts a run already writes: the repository end state, the diff against a baseline commit, and the run transcript. Re-scoring a published record requires no API key, no network, and no trust in the authors.
- **A closed assertion vocabulary and declarative tasks.** Tasks are JSON, not code. This trades expressive power for the property that any two tasks are comparable and any grading rule is reviewable without reading a checker.
- **Refusal as a graded, ungameable behavior.** Tasks declare an expected outcome class, and the suite mixes classes so that refusing everything and refusing nothing both score poorly. Refusals are graded on whether they cite the governing budget, not on whether they sound careful.
- **Cost as a reported axis.** We report cost per *passing* task rather than per run, since a model that fails cheaply is not cheap, and we segment providers by accounting fidelity rather than averaging incomparable cost figures into one column.

- **A deterministic failure taxonomy that ranks work.** Failures are classified without a model into eleven categories, ranked by frequency times mean cost, which turns a benchmark result into a queue of things to fix.
- **A generated-evidence discipline for the paper itself.** Every figure here is emitted from the released records, and a hand-typed number in a results section fails the build.

1.2 What this benchmark does not claim

A passing score is not a statement that a board can be manufactured, that a part choice is wise, or that a layout is good. The oracle establishes legality and self-consistency. Results are also conditioned on one harness: a model that scores poorly here may perform differently under different prompts, tools, and gates, and that is a finding about the harness as much as about the model. Section 9 states these limits and the rest.

2 Related work

Agentic software benchmarks. SWE-bench evaluates language models on real GitHub issues, using the repository’s own tests as the oracle [Jimenez et al., 2024]; SWE-agent studies the interface between an agent and a repository [Yang et al., 2024]. Earlier function-level benchmarks such as HumanEval established execution-based grading for self-contained problems [Chen et al., 2021]. We inherit the execution-based stance and the use of a real repository as the working surface. We depart on the oracle: no artifact in a KiCad project states what a requested change was supposed to accomplish, so the intent half of grading has to be authored per task rather than harvested from the repository. That authoring is what the closed assertion vocabulary constrains.

Language models for hardware description languages. A substantial line of work evaluates models on Verilog and related HDLs: VeriGen and its benchmark of RTL generation tasks [Thakur et al., 2023], VerilogEval [Liu et al., 2023], and RTLLM [Lu et al., 2024]. Chip-Chat explored conversational hardware design with a human in the loop [Blocklove et al., 2023]. These share our domain but not our artifact. HDL is text that compiles and simulates, which restores much of the software oracle; a schematic and a board are structured design databases whose correctness conditions are geometric, electrical, and documentary. They also evaluate generation from scratch, where we emphasize edits to an existing project, which is where propagation, surgicity, and constraint retrieval become measurable.

Tool-using agents. The tool-use loop we measure descends from ReAct-style interleaving of reasoning and action [Yao et al., 2023]. Our concern is narrower: not whether a model can use tools, but what it does when the tools include a verifier that can reject its work and a registry of constraints that can make the request impossible.

Positioning. To our knowledge copperbench is the first benchmark that evaluates language-model agents on edits to real KiCad projects, grades them with electrical verification plus declarative end-state assertions, treats refusal as a scored outcome class with its own correctness condition, and reports cost per passing task as a headline figure alongside pass rate.

Note for the preprint: bibliography entries in `references.bib` are marked `VERIFY` and must be checked against the canonical records before submission.

3 The system under test

copperbench measures models inside *copperhead*, an open-source command-line agent that edits KiCad projects, maintains markdown design documents as memory, and verifies its own work with *kicad-cli*. Three properties of that harness shape what the benchmark can observe, and are stated here because results are conditioned on them.

Edits are anchored text replacements, never regeneration. The agent reads `.kicad_sch` and `.kicad_pcb` files through a parser that never serializes. A change is an exact-match replacement against surrounding context. This is what makes a diff-shape assertion meaningful: blast radius is a property the harness is trying to keep small, so measuring it measures something the agent controls.

Mutations are gated on verification. No change is complete until electrical rule checking passes, and design rule checking if the board changed. Violations trigger a bounded repair loop; if they persist, the run rolls back to a snapshot taken before it started. A benchmark run therefore terminates in one of a small set of machine-readable exit paths, and the distinction between “failed to converge” and “converged on the wrong thing” is available in the record rather than inferred.

Constraints are recorded, dual-written, and consultable. Budgets stated in design documents are mirrored into a machine-readable registry. This is what makes the refusal tasks gradable: there is a specific key an agent must find and cite, and a specific file it must not quietly edit to make a request legal. Without a registry, “did it respect the budget” would be a judgment call.

Each run writes a redacted JSONL transcript containing a metadata block at start and a statistics block at end: exit path, turns used against budget, repair cycles, per-turn token counts, and wall-clock duration. Those blocks are the third evidence source in Section 4, and their pre-existence is why the benchmark needs no instrumentation of its own.

4 Benchmark design

4.1 Two constraints that generate the design

Grading must not call a model. Running a task needs a provider and costs money. Grading it must not, because a score that can only be reproduced by parties who hold credentials and are willing to spend is not a score anyone can check. Everything a verdict depends on is recomputable offline from artifacts the run already wrote.

A task must be data. Adding a task means adding JSON and prose. If grading a task would require new code, the check either enters the shared vocabulary or the task does not enter the suite. This is a real loss of expressive power, accepted for a specific gain: two tasks graded by two bespoke checkers are not comparable in principle, and their grading rules cannot be reviewed without reading the checkers.

4.2 The unit of evaluation

A task is a directory containing a manifest, an assertion list, and a prose statement of what it tests. The manifest declares the difficulty tier, the copperhead surface to drive, a content-hashed fixture project, the request, the expected outcome class, configuration overrides including recorded budgets, and hard caps on turns and wall-clock time. A run is one execution of one task by one model in its own sandbox; a repeat is one of several runs of the same pair.

Each run is materialized fresh: the pinned fixture is copied outside the source repository, a git baseline is committed, the response cache is disabled, and the run proceeds from a clean tree. Repeats never share a sandbox. The cache is disabled because a replayed turn would manufacture determinism the model does not possess, and the tree starts clean because rollback assertions are only meaningful relative to a known good state.

4.3 Three evidence sources

An assertion may read the repository end state, the diff against the baseline commit, or the run transcript. Nothing else. Console output is explicitly excluded: it is presentation, free to change without a behavioral change, and grading against it would score formatting edits as regressions. All three admissible sources are files on disk, which is what makes offline re-scoring possible.

4.4 The assertion vocabulary

Grading rules are drawn from a closed, typed vocabulary spanning the three sources: verification outcomes (electrical and design rule checks, full-project check, document drift), structural end-state queries (net and symbol presence, pin-to-net assignment, document table cells, constraint registry entries), diff-shape checks (permitted file set, file untouched, bounded change ratio, commit count, byte-identical rollback, secret absence), and transcript checks (exit path, refusal citation, event occurrence). An assertion type outside the vocabulary is a validation error rather than a skipped check, and adding a type bumps the suite version.

Two entries deserve comment. The bounded change ratio turns edit surgicality from a review opinion into a threshold: a regenerated schematic can satisfy every connectivity and document assertion a rename task states, and this is the only check that fails it. The refusal citation check grades a refusal on whether it names the governing budget key, not on whether it reads as cautious, because cautious prose is exactly what a model can produce without performing the arithmetic.

4.5 Outcome classes and why refusal is ungameable

Every task declares one expected outcome: *edit*, where a change satisfying all recorded constraints exists; *refusal*, where none does and the correct response cites the arithmetic; or *flag*, where the repository is inconsistent in a way that reflects a requirement violation and the correct response surfaces both sides without resolving either.

The suite contains all three classes, and this is structural rather than decorative. A model that refuses everything fails every edit task. A model that never refuses fails every refusal task. Correct-refusal rate and false-refusal rate are reported separately because they are different defects with different remedies.

4.6 Scoring

A task passes only when every assertion marked required passes. Weighted partial credit is computed and recorded but never reported as a headline, since a headline that rewards partial progress rewards touching files. Two headline figures are reported, always broken out by tier: strict pass rate, and cost per *passing* task. The latter is deliberate: cost per run flatters a model that fails cheaply, and the practical question a user asks is what a completed task costs.

4.7 Repeats

Each pair is run several times by default. We report pass@1 as the mean across repeats, which is the experience of a single user run, and pass^k as the fraction of tasks passing every repeat, which is reliability. The gap between them is itself a result: a model that succeeds two times in three is a different product from one that succeeds consistently, and reporting only the mean conceals that. A report derived from a single repeat is labeled as such and is not published as a comparison.

4.8 Cost accounting

Cost is computed from a dated, versioned price table, and every record stores both the resulting figure and the table version, so a later price change cannot rewrite history. Providers are segmented by accounting fidelity: direct API models report exact cost; hosted open-weight endpoints report their endpoint price; self-hosted runs report no cost figure, because the honest unit is accelerator-hours rather than tokens; and saved-login command-line routes report no cost and carry a weak-pinning flag, since the model behind such a login can change without notice. These segments are tabulated separately. Averaging them into one currency column would produce the most misleading table this benchmark is capable of.

4.9 Comparability

Every record carries a stamp: schema version, suite version, task manifest hash, fixture hash, harness version and commit, and tool versions. Records merge into a shared table only when those agree, and disagreeing records render in a separate labeled section rather than being averaged in. Any edit to a task, fixture, or the vocabulary bumps the suite version. Re-running a full matrix on every suite edit is unaffordable, which is precisely why the segregation is mechanical instead of remembered; because grading is deterministic and offline, preserved runs can be re-scored under a new suite version at no provider cost.

4.10 Failure taxonomy

Each failed run is classified into exactly one of eleven categories, derived from the exit path, the first failed required assertion, and the dominant tool-error category in the transcript, with no model involved. The categories separate plumbing failures (malformed tool calls, reverted unloadable edits, exhausted turn or repair budgets, stalls, commit failures) from behavioral ones (unmet obligations, left-behind document drift, constraint violations, false refusals) and from the category that matters most: a run that verified, committed, and did not make the requested change. That last category is easy to lose, because from every angle except the end-state assertion it resembles a success.

Categories are ranked by frequency times mean run cost. That ranking is the intended output of a benchmark run for a maintainer: not a score, but an ordered list of what to fix and roughly what fixing it is worth.

4.11 Measuring memorization

The fixtures, briefs, and this standard are public and will be trained on. For a declared subset of tasks the suite ships mutated variants generated by a deterministic transform: renamed nets and reference designators, permuted pin assignments, and scaled budget figures that preserve the reasoning while defeating recall. The variant gap, the difference in pass rate between originals and variants, is reported per model, which converts a standing caveat into a measurement.

5 Experimental setup

All figures in this section and those following are generated from the released result records; see Section 10.

Provenance. Suite version 0.1.0-draft, result snapshot none (0000000), 0 runs over 2 tasks and 0 models at 3 repeats per pair. Harness version 0.8.1, `kicad-cli` unrecorded.

Model matrix. This revision covers none. The following segments are absent and their rows are labeled accordingly throughout: `api-frontier`, `api-cheap`, `open-weight`, `saved-login`. The intended full matrix spans four segments, which differ in what can be known about them rather than only in capability:

- **Direct API, frontier tier.** Exact model identifiers, exact token accounting, exact cost.
- **Direct API, cheap tier.** Same accounting. Included because cost per passing task is uninteresting without a spread in price, and because the verification loop may or may not rescue a weaker model.
- **Open-weight.** Hosted endpoints report endpoint pricing; self-hosted runs report tokens without a currency figure.
- **Saved-login command-line routes.** Reported separately with weak model pinning and no cost figure. Included because a substantial share of real usage runs this way, and excluding it would measure a population we do not have.

Suites. The default suite covers edit-mode and check-mode tasks and is bounded per task by turn and wall-clock caps. The full suite adds create-mode tasks, which run a multi-stage pipeline from a product brief and cost substantially more per run; create-mode tasks may declare a lower repeat count, recorded per result.

Environment. Runs execute against a pinned reference environment; the tool versions above are part of the comparability stamp. Results produced in a different environment remain valid evidence about that environment and are not merged into these tables.

6 Results

6.1 Headline: pass rate and cost per passing task

No results snapshot bound. Run the suite and regenerate.

6.2 Consistency across repeats

Pass@1 reports the expected outcome of a single user run; pass^k reports the fraction of tasks that passed every repeat. The gap between them is the reliability story.

No results snapshot bound.

6.3 Refusal correctness

Correct-refusal rate is measured over tasks whose expected outcome is a refusal; false-refusal rate is measured over tasks with a correct edit. A model cannot improve one without exposure on the other, which is the point of mixing the classes.

No results snapshot bound.

6.4 Surgicity

Blast radius on edit tasks, measured as changed lines relative to the baseline file and as the set of files touched outside the permitted set.

No results snapshot bound.

6.5 Memorization: original versus mutated variants

No results snapshot bound.

6.6 Weakly pinned routes

Saved-login command-line routes are reported here rather than in the headline table: their underlying model may change without notice and their token accounting is not under the harness’s control, so neither model identity nor cost is comparable to the direct API rows.

No results snapshot bound.

7 Failure analysis

Every failed run is classified into exactly one category by a deterministic rule over the exit path, the first failed required assertion, and the dominant tool-error category in the transcript. The classification involves no model, so it is reproducible offline alongside the verdicts themselves.

7.1 Categories ranked by frequency times mean cost

No results snapshot bound.

7.2 Reading the taxonomy

The categories separate three kinds of problem, and the separation is the useful part.

Plumbing. Malformed tool calls, reverted unloadable edits, exhausted turn or repair budgets, stalls, and commit failures indicate the harness is spending the model’s turns badly. These are the cheapest failures to fix and rarely say much about the model.

Behavior. Unmet synchronization obligations, left-behind document drift, constraint violations, and false refusals indicate the model understood the mechanics and got the judgment wrong.

Silent wrongness. A run that passed verification, committed, and did not make the requested change is the category this benchmark exists to surface. It is invisible to every check the project can run on itself, and it is the failure mode with the highest cost outside the benchmark, where the next observation point is a fabricated board.

7.3 Worked examples

To be populated from the bound snapshot.

8 Ablations

Ablations are recorded overrides on the same tasks rather than forked task copies: prompt variant, turn budget, tool subset, and repair-cycle cap are stamped into each result record. Duplicating a task per condition would multiply the suite and destroy the same-task-across-conditions comparison that makes an ablation informative.

Three questions motivate the conditions we run.

Does the verification loop rescue weaker models? If a cheap model reaches the same pass rate as a frontier model given more repair cycles, the interesting quantity is what that costs, and whether the extra cycles converge or merely churn. Varying the repair-cycle cap separates a model that fixes its own violations from one that resubmits them.

Does the turn budget bind? Runs that end at the turn cap are not necessarily incapable; they may be inefficient. Raising the cap distinguishes a model that would have finished from one that was not converging, and the distinction changes whether the fix belongs in the prompt or in the model choice.

Do prompt changes generalize across models? A prompt tuned against one model can regress others. Running a variant across the matrix is the only way to tell an improvement from an overfit, and it is the reason ablation results are reported per model rather than pooled.

Conditions and results are populated from the bound snapshot.

9 Threats to validity and limitations

The oracle is incomplete. Passing electrical and design rule checks, and agreeing with the project’s own documents, establishes that a design is legal and internally consistent. It does not establish that the board can be manufactured, that the part selection is sound, that the layout is good, or that the design meets any requirement not encoded as a recorded budget or an assertion. No result here should be read as a fabrication-readiness claim.

Intent is authored, not harvested. Unlike a software benchmark that inherits an oracle from a repository’s tests, our end-state assertions are written by us. A task’s assertions are an interpretation of the request. We constrain that interpretation with a closed vocabulary and publish every assertion, which makes disagreement possible rather than impossible, but it does not make the interpretation neutral.

Contamination. The fixtures, briefs, and this document are public and will appear in training data. We measure rather than assume the effect: mutated variants preserve required reasoning while defeating recall, and the variant gap is reported per model. The mitigation is partial. A transform we designed may be inverted by a model that has seen the transform described here.

Repeats are few and models are non-deterministic. Agent runs vary even at low temperature, because tool results, ordering, and timing vary. Our repeat count bounds cost, not variance. Consequently we report pass^k and spread alongside means, treat single-run figures as unpublishable, and set a threshold floor on the regression gate rather than reacting to any movement.

Models change under their names. A model identifier can refer to different weights over time, and saved-login routes can change the underlying model without any identifier changing at all. Records stamp what was knowable at run time; for weakly pinned routes what was knowable is less. Those rows carry the flag and are never merged into the shared cost column.

Results are conditioned on one harness. Every score is a score for a model inside copperhead’s prompts, tools, and gates. A model that scores poorly here may do well elsewhere, and improvements to the harness will move every model’s score. Comparisons across suite versions are segregated for this reason.

Cost figures are not fully commensurable. Exact for direct API models, endpoint-dependent for hosted open-weight models, absent for self-hosted and saved-login routes. Cost per passing task is comparable within a segment and should not be read across segments.

Tier assignment is a judgment. Difficulty tiers reflect how much a task requires an agent to hold at once, assigned by us. They are a reporting convenience, not a measured quantity, and a reader who disagrees with a tier assignment can recompute the tables from the released records with a different grouping.

Rubric results, where reported, are opinion. The model-judged tier is disabled by default and reported in a labeled column. It contributes to no headline figure and no gate. Where it appears, it reflects one judge model’s reading of a rubric and inherits every weakness of model-based evaluation.

Fixture provenance is concentrated. Fixtures must be real boards under permissive licenses in modern KiCad format, and that intersection is narrow: reciprocally licensed and legacy-format designs are excluded by policy, which removes much of the published open-hardware corpus. At the time of writing our fixtures come from a small number of upstream origins, so results partly reflect how well models handle those organizations’ design conventions. Broadening origin diversity is a prerequisite for treating the suite as representative of open hardware generally.

Fixtures carry baseline warnings. Real boards in a bare checkout cannot resolve their vendor libraries, so verification is asserted relative to a recorded baseline rather than absolutely. This is necessary but weakens the oracle at warning severity: a run that introduces a new warning of a type already present in the baseline is not distinguished from one that introduces none. Error-severity violations, unconnected items, and schematic parity are unaffected, since fixtures with any of those at baseline are rejected.

Scope of the suite. The current suite emphasizes edits to existing projects and a small number of end-to-end pipeline tasks. Layout quality, thermal and signal-integrity reasoning, multi-board systems, and manufacturability are not represented.

10 Reproducibility

We distinguish two claims, because they are not equally strong and conflating them is the usual way agentic results become unfalsifiable.

Re-scoring, which we claim fully. Given the released records, anyone can recompute every assertion outcome, every verdict, and every table in this paper, with no provider credential and no network access:

```
git clone https://github.com/chouhanindustries/copperbench
cd copperbench && npm install
npm run benchmark -- --rescore results/<snapshot>
npm run paper
```

Identical assertion outcomes and a byte-identical regenerated set of tables is the reproduction claim. It holds because grading reads only files the runs already wrote, and involves no model.

Re-running, which we claim weakly. Executing the suite again requires credentials, costs money, and will not reproduce these numbers exactly. The models are non-deterministic, and the weights behind a model identifier may have changed. A re-run is evidence about the models as they are at the time of the re-run, which is a different and also useful thing.

Released artifacts. The task suite, fixtures, assertion schemas, result records, runner, scorer, and the source of this paper are released under Apache-2.0 in the project repository. Each record names the suite version, task and fixture hashes, harness version and commit, and tool versions needed to interpret it in isolation.

Generated figures. Every quantitative claim in Sections 5 through 8 is emitted from the bound snapshot into generated LaTeX. A checker rejects literal numerals in those sections, so a hand-typed figure fails the build rather than surviving into a preprint. This is the same discipline the system under test applies to design documents, applied to the document that reports on it.

11 Conclusion

Hardware design gives an agent benchmark a strong verifier and no statement of intent. Everything in copperbench follows from taking both halves of that seriously: verification is used where it is authoritative, end-state assertions supply what it cannot say, and the two are graded separately so that a legal design and a correct one are never confused.

Three choices are worth carrying into other agentic domains with the same shape. Grading offline and model-free makes a published number checkable by parties who do not trust the authors, and costs little once the runs write structured artifacts. A closed grading vocabulary trades expressive power for comparability and reviewability, and the trade is favorable as soon as a suite has more than a few authors. Scoring refusal as an outcome class, with a citation requirement and a task mix in which neither compliance nor caution wins, turns a safety property into something measurable rather than asserted.

The benchmark’s most useful output is not the leaderboard. It is the ranked failure taxonomy: an ordered list of what to fix, priced by what fixing it is worth. We expect the scores here to age quickly. We expect the standard, and the discipline of generating every reported number from released evidence, to age better.

References

- Jason Blocklove, Siddharth Garg, Ramesh Karri, and Hammond Pearce. Chip-Chat: Challenges and opportunities in conversational hardware design. In *ACM/IEEE Workshop on Machine Learning for CAD (MLCAD)*, 2023. VERIFY: arXiv:2305.13243.
- Mark Chen, Jerry Tworek, Heewoo Jun, et al. Evaluating large language models trained on code. *arXiv preprint*, 2021. VERIFY: arXiv:2107.03374.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024. VERIFY: arXiv:2310.06770.
- Mingjie Liu, Nathaniel Pinckney, Brucek Khailany, and Haoxing Ren. VerilogEval: Evaluating large language models for Verilog code generation. In *IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2023. VERIFY: arXiv:2309.07544.
- Yao Lu, Shang Liu, Qijun Zhang, and Zhiyao Xie. RTLLM: An open-source benchmark for design RTL generation with large language model. In *Asia and South Pacific Design Automation Conference (ASP-DAC)*, 2024. VERIFY: arXiv:2308.05345.
- Shailja Thakur, Baleegh Ahmad, Zhenxing Fan, Hammond Pearce, Benjamin Tan, Ramesh Karri, Brendan Dolan-Gavitt, and Siddharth Garg. Benchmarking large language models for automated Verilog RTL code generation. In *Design, Automation and Test in Europe Conference (DATE)*, 2023. VERIFY: arXiv:2212.11140.
- John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. VERIFY: arXiv:2405.15793.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023. VERIFY: arXiv:2210.03629.

A Task suite

Generated from the bound suite version.

B Assertion vocabulary

Generated from the bound suite version.